Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

GAPFRUIT

Semester Project

Firmware Updater for Infineon TPMs

Jan Häussermann
ETH Zürich

Supervisors: Alice Domage, Stefan Thöni (Gapfruit)
Professor: Shweta Shinde (ETH Zürich)
Date: August 2026

Contents

1 Introduction **4**
- 1.1 Motivation . . . 4
- 1.2 Problem Statement . . . 4
- 1.3 Goals and Scope . . . 4
- 1.4 Report Structure . . . 4

2 Background **6**
- 2.1 Trusted Platform Module (TPM) . . . 6
- 2.2 TPM 2.0 Command Interface . . . 6
  - 2.2.1 TPM2 TSS Software Stack . . . 7
- 2.3 Infineon SLB9670 and SLB9672 . . . 7
- 2.4 Gapfruit OS and Genode . . . 7

3 Firmware Update Protocol and Binary Format **8**
- 3.1 Overview . . . 8
- 3.2 TPM Information Required for Firmware Updates . . . 8
  - 3.2.1 Standard TPM Properties . . . 9
  - 3.2.2 Infineon Vendor-Specific Properties . . . 10
  - 3.2.3 SLB9670 Security Module Information . . . 10
  - 3.2.4 Device Identification . . . 11
- 3.3 Firmware Update Package . . . 11
  - 3.3.1 Firmware Reader . . . 12
  - 3.3.2 Package Identification . . . 12
  - 3.3.3 Firmware Image Versions . . . 12
  - 3.3.4 Firmware Image Structure . . . 13
  - 3.3.5 Manifest and Policy Data . . . 14
  - 3.3.6 Firmware Payload . . . 14
- 3.4 Firmware Blob Parsing . . . 14
  - 3.4.1 Serialized Layout . . . 14
  - 3.4.2 Variable-Length Data . . . 15
  - 3.4.3 SLB9670 Legacy Policy Data . . . 15
  - 3.4.4 Byte Order and Bounds Checking . . . 16
- 3.5 Firmware Update Commands . . . 16
  - 3.5.1 Command and Response Format . . . 16
  - 3.5.2 SLB9670 Commands . . . 17
  - 3.5.3 SLB9672 Vendor Commands . . . 17
  - 3.5.4 Manifest Block Processing . . . 17
- 3.6 Firmware Update State Machine . . . 17
  - 3.6.1 SLB9670 States . . . 18
  - 3.6.2 SLB9672 Operation Modes . . . 18

4 Design **19**
- 4.1 Overview . . . 19
- 4.2 TPM Communication . . . 19
  - 4.2.1 TPM Connection Context . . . 19
  - 4.2.2 TPM Information Context . . . 19
- 4.3 Firmware Parser Design . . . 20
  - 4.3.1 Parsed Firmware Representation . . . 20
  - 4.3.2 Firmware Reader . . . 20
  - 4.3.3 Validation Strategy . . . 20
- 4.4 Firmware Updater Design . . . 21
  - 4.4.1 Update Interface . . . 21
  - 4.4.2 Device-Specific Update Paths . . . 21
  - 4.4.3 Update State Handling . . . 21
- 4.5 Error and Recovery Handling . . . 21
- 4.6 Security Considerations . . . 23

5 Implementation **24**
5.1 Component Implementation 24
5.1.1 TPM Connection Context 24
5.1.2 TPM Information Retrieval 24
5.2 Firmware Parser 25
5.3 TPM Command Serialization 25
5.4 SLB9670 Support 26
5.5 SLB9672 Support 26
5.6 Integration into Gapfruit OS 27

6 Evaluation **28**

7 Conclusion **29**
7.1 Summary of Results 29
7.2 Limitations 29
7.3 Future Work 29

Abstract

Trusted Platform Modules (TPMs) provide a hardware root of trust. The TPM is used for secure boot, platform integrity, and trusted device identities. For that reason, the TPM is a security-critical component that needs to be maintained through its lifetime. Previously disclosed vulnerabilities in CPU and platform integrated firmware TPM implementations, such as Intel's Converged Security and Management Engine and Server Platform Services [1, 2] and AMD's firmware TPMs [3], showed that weaknesses in TPM implementations can weaken security guarantees. Therefore, securely updating TPM firmware is an essential part of maintaining a trustworthy system.

Gapfruit OS uses TPMs from Infineon, mainly the SLB9670 and SLB9672. They provide the hardware root of trust. Infineon provides firmware update packages and reference implementations for these devices. The issue with the supplied update mechanisms is that they target mainstream operating systems such as Linux. For that reason, they cannot be directly applied to Gapfruit OS, a micro-kernel based OS built with the Genode Operating System Framework.

The goal of this project is to implement the firmware updater for the Infineon SLB9670 and SLB9672 on Gapfruit OS, following Infineon's reference firmware updater implementation for Linux based distros. The reference implementation is analyzed to identify the commands, data structures, update sequence, and states that are involved in the update sequence. Based on these findings, a Genode component is designed and implemented to perform the TPM firmware update.

The implementation is tested on development boards equipped with SLB9670 and SLB9672 TPMs, where firmware updates are successfully performed on Gapfruit OS. This enables TPM firmware to be maintained throughout the lifetime of the device.

1 Introduction

Modern systems rely on hardware supported security mechanisms to establish trust in the software stack. Trusted Platform Modules (TPMs) are commonly used as a hardware root of trust and provide functionality for secure boot, platform integrity, and device identity. Some modern operating systems, such as Windows 11, require a TPM for normal operation. Since the TPM is a processor, it runs its own firmware. For that reason, the firmware has to be maintained throughout its lifetime to keep its security guarantees.
Gapfruit OS uses Infineon TPMs as part of its trusted computing infrastructure. While Infineon provides a firmware updater for its TPMs, it is designed for mainstream operating systems such as Linux or Windows. This semester project explores the Infineon updater for the SLB9670 and SLB9672 TPMs and implements the updater in Gapfruit OS.

1.1 Motivation

TPMs provide a higher level of security guarantees through their hardware root of trust. These guarantees can be weakened when the TPM firmware has vulnerabilities. Previously disclosed vulnerabilities affected firmware based TPM implementations from AMD and Intel, which needed firmware updates to fix them. For that reason, TPMs should not be seen as fixed hardware components that never need maintenance, but as independent systems that need updates when vulnerabilities are found.
For the reasons mentioned above, firmware updates are an important part of device lifecycle maintenance. This is especially relevant for systems that are expected to remain deployed for long periods of time. Gapfruit uses TPMs throughout all stages of a device lifecycle. Being able to update TPM firmware directly on Gapfruit OS allows the TPM firmware to be maintained without relying on other operating systems.

1.2 Problem Statement

Infineon provides a firmware updater for the SLB9670 and SLB9672 TPMs. The TPM firmware updater relies on functionality and interfaces available in these environments. Gapfruit OS is based on the Genode Operating System Framework and uses its own architecture.
Therefore, the Infineon reference updater cannot be used directly on Gapfruit OS. The firmware update protocol, firmware package format, and vendor specific update sequences first need to be understood. Based on this information, a firmware updater can be created for Gapfruit OS and integrated with the existing TPM infrastructure.
A challenge is that the SLB9670 and SLB9672 use different update sequences and different vendor specific commands. The implementation should therefore support both TPM families while sharing functions where possible.

1.3 Goals and Scope

The goal of this project is to implement a firmware updater for the Infineon SLB9670 and SLB9672 on Gapfruit OS. The following tasks are part of this project:

- analyze the Infineon firmware updater to understand the update procedure and binary structures used
- identify the differences between the SLB9670 and SLB9672
- implement a TPM information getter
- implement a firmware update package parser
- implement the update procedure
- run firmware updates on both TPMs

1.4 Report Structure

The remainder of this report is structured as follows. Chapter 2 introduces the background required to understand TPMs, the TPM 2.0 command interface, and the Infineon SLB9670 and SLB9672. Chapter 3 analyzes the Infineon firmware update protocol, firmware package format, update commands, and device specific state machines. Chapter 4 presents the design of the firmware updater,

including TPM firmware parsing, validation, and update state handling. Chapter 5 describes the implementation and its integration into Gapfruit OS. Chapter 6 evaluates the implementation on development hardware. Finally, Chapter 7 summarizes the results, discusses limitations, and presents possible future work.

2 Background

This chapter explains background information that is required to understand the TPM firmware update in this project. First, the Trusted Platform Modules and their tasks are explained. Afterwards, the TPM 2.0 standard and the software stack are introduced. The Infineon SLB9670 and SLB9672, which are the targeted TPMs in the project, are introduced. In the end, Gapfruit OS and Genode are described. All this information provides the required background to understand the chapters that follow.

2.1 Trusted Platform Module (TPM)

A Trusted Platform Module is a secure cryptographic processor that provides hardware cryptographic functions. The TPMs used in this project are separated from the main processor and can be used by the operating system and applications without exposing sensitive data.
One important functionality is the protected storage and use of cryptographic keys. The TPM can create and use keys within itself without exposing them outside the chip. Such keys can be exported, always in an encrypted form, rendering them unusable outside the TPM. This reduces the consequences of an attack on the system memory. The TPM also provides functionality for random number generation, hashing, signing, attestation and other operations such as PCR banks and policies.
The TPM has non volatile storage. This allows data to be stored inside the TPM and kept across restarts.
Key usage and NVRAM access can be controlled with authorization. A TPM policy is a mechanism in the TCG TPM specification that enforces the conditions under which a key or NVRAM index may be used, such as requiring specific PCR values, a particular locality, a time window, or a command specific approval before the TPM allows a specific cryptographic operation to be performed.
The TPM itself is a processor and runs its own firmware. The firmware implements the commands and controls the internal functionality of the device. Hence, it should not be treated as hardware that is static, because vulnerabilities can be found in the firmware which require firmware updates to maintain the security guarantees.
This is especially important for systems with a long lifetime. A device may stay deployed for many years while vulnerabilities are discovered. During this time, updating the TPM firmware becomes part of the maintenance of the system.
The focus of this project is on TPMs implementing the TPM 2.0 specification. The specification defines the architecture, data structures, and commands used to interact with a TPM. Because Gapfruit OS mainly uses TPMs from Infineon, specifically the SLB9670 and SLB9672, the firmware updater is designed for those TPMs.

2.2 TPM 2.0 Command Interface

Communication with the TPM is done via a command and response interface over an SPI bus. A command requests an operation from the TPM device. Depending on the command, additional parameters and authorization information are required to be sent with the request. The TPM then processes the request and returns a response.
The basic command and response formats are defined by the TPM 2.0 specification [4, 5, 6]. Every command contains a header, which contains a tag, the size of the command, and the command code. The command code defines which operation the TPM should execute. Depending on the command, the additional parameters follow the header.
Because sensitive information that is part of a command may be transmitted over an unencrypted bus from the host CPU to the TPM, the TPM 2.0 specification describes a parameter encryption mechanism to guarantee the confidentiality of that information. This mechanism is set up at a session level by the software interacting with the TPM.
The response has a similar structure. It starts with the response header with the response size and response code. When the request is successful, additional parameters can be returned. The response code indicates if the command was executed successfully. If the command fails, the response code contains information about the failure.
The TPM 2.0 specification defines standardized command codes for the normal TPM functionality. An important command for the firmware updater is `TPM2_GetCapability`. It returns information from the TPM such as the manufacturer, firmware version, and vendor specific information.

Most information used by the firmware updater is standardized by TPM 2.0, but some information is vendor specific. The Infineon SLB9670 and SLB9672 also use vendor specific commands which are not defined by the TPM 2.0 specification. This is important because standardized functions can be accessed through existing TPM software libraries. For the vendor specific functionality, the format has to be the one that Infineon expects.

2.2.1 TPM2 TSS Software Stack

Instead of building a TPM software stack, two libraries are used to communicate with the TPM. The Enhanced System API (ESAPI/ESYS) and the TPM Command Transmission Interface (TCTI) are used to communicate with the TPM.

TPM Command Transmission Interface (TCTI) The purpose of the TPM Command Transmission Interface (TCTI) is to transport the commands to the TPM and receive the response from the TPM. It does not implement TPM commands and is solely in charge of the transfer. This is useful for implementations that focus on the commands and not the transport, such as ESAPI. The TCTI implementation stores its state inside a structure which is required for communication.

Enhanced System API (ESAPI/ESYS) The Enhanced System API (ESAPI/ESYS) is a higher level library that provides functions for the TPM 2.0 standard. It provides functions to get random numbers, information from the TPM, and more. The library needs the TCTI state structure during initialization because it uses it as a transport layer to the TPM. Using ESAPI reduces the amount of code that needs to be maintained in the firmware updater while keeping a coherent API with respect to the command interface described in the TCG TPM2 specification. Higher level libraries exist, but their level of abstraction is not suitable for advanced use of the TPM2 interface. Infineon's vendor specific commands still need to be manually implemented.

2.3 Infineon SLB9670 and SLB9672

Two families of TPMs are relevant for this project: the SLB9670 and SLB9672, which Gapfruit OS relies on for secure edge gateway applications. Both TPMs implement TPM 2.0 and communicate over an SPI interface [8, 9]. The TPMs are different generations, which is noticeable in the update mechanism and the vendor specific commands available. The SLB9670 has one large vendor specific structure that contains information about the TPM, whereas the SLB9672 has multiple smaller vendor specific properties to retrieve information from the TPM.
The exact differences between the two TPMs are not the main focus of the project. The only relevant differences are the vendor specific properties and commands used to receive information and the update sequences of the TPMs. No in- depth analysis of the two TPM families is required.

2.4 Gapfruit OS and Genode

Gapfruit OS is based on the Genode Operating System Framework. Genode uses a component based architecture where the functionality is separated into components and services. A component has access only to the exact resources and services it really needs. This differs from operating systems such as Linux. There is also no global namespace every component can access. Instead, each component is given a set of capabilities to access other components' services.
The Infineon reference firmware updater is designed for operating systems such as Linux. It depends on interfaces and functionality available in these environments and cannot be applied to Gapfruit OS. For that reason, the implementation has to use the interfaces available in Gapfruit OS while still following the update procedure expected by the Infineon TPM.

3 Firmware Update Protocol and Binary Format

3.1 Overview

The firmware update involves more than transferring the new firmware image to the TPM. Before the transfer can start, the updater has to identify the TPM, read its properties, parse the firmware package, and check compatibility between the firmware image and the TPM. The SLB9670 and SLB9672 implement the TPM 2.0 standard, but use different update mechanisms. The SLB9670 mainly uses an older Infineon field upgrade mechanism and returns TPM information back in one large vendor specific data structure. The SLB9672 uses newer vendor specific properties to retrieve the required values, and its update sequence is different as well.
The implementation is based on the Infineon reference updater [7], and the required parts are reimplemented for Gapfruit OS. The main Infineon sources used in this chapter are
`Source/Common/FirmwareImage.h`, `Source/Common/FirmwareImage.c`,
`Source/Common/FirmwareUpdate.c`,
`Source/Common/MicroTss/Tpm_2_0/TPM2_FieldUpgradeTypes.h`, and the vendor command implementations in `Source/Common/MicroTss/Tpm_2_0/TPM2_FieldUpgrade*Vendor.c`. The project does not copy the complete Infineon updater. It focuses on the data structures and protocol fields that are required for the SLB9670 and SLB9672 and implements the parsing and update mechanism using interfaces available in Gapfruit OS.
This chapter describes the information that has to be read from the TPM. Afterwards, the firmware package and the firmware image format are described. The parser is then explained, including byte order, variable length fields, and bounds checking. Finally, the firmware update commands and device specific state machines are described.

3.2 TPM Information Required for Firmware Updates

The updater collects information about the TPM in the `TPMInformation` structure. It is used internally by the updater to combine all required TPM properties for the validation and update functions.

```
struct TPMInformation{
    uint16_t upgrade_counter;
    uint16_t upgrade_counter_same;
    uint64_t firmware_version;
    uint32_t manufacturer;
    uint64_t type;
    uint16_t operation_mode;
    uint16_t max_block_size;
    uint32_t target_hardware_identifier;
    uint16_t decrypt_key_count;
    uint32_t decrypt_key_ids[4];
    uint32_t active_firmware_version;
    uint8_t  process_firmware_unique_id;
    uint32_t key_group_id;
    uint32_t firmware_update_properties;
    uint16_t start_hash_algorithm;
    uint16_t start_hash_size;
    uint8_t  start_hash_digest[64];
    uint8_t  vendor_string[16];
};
```

Figure 1: TPM information structure

Variable	Type	Meaning and use
`upgrade_counter`	`uint16_t`	Number of firmware upgrades still available on the TPM.

Variable	Type	Meaning and use
upgrade_counter_same	uint16_t	Number of firmware upgrades still available for the same TPM version. It is reset when updating to a new version and is only available on the SLB9672.
firmware_version	uint64_t	Current TPM firmware version that is running on the device.
manufacturer	uint32_t	TPM manufacturer identifier.
type	uint64_t	Identifies the TPM type.
operation_mode	uint16_t	Current firmware update state of the TPM. The meaning depends on the TPM.
max_block_size	uint16_t	Maximum firmware data size that can be transferred to the SLB9670 in one block.
target_hardware_identifier	uint32_t	Hardware identifier reported by the SLB9670. It is used to check whether the image targets the connected TPM.
decrypt_key_count	uint16_t	Number of valid firmware decryption key identifiers reported by the SLB9670.
decrypt_key_ids	uint32_t[4]	Firmware decryption key identifiers supported by the SLB9670.
active_firmware_version	uint32_t	Firmware package version currently active on the SLB9670.
process_firmware_unique_id	uint8_t	Identifier used when checking a firmware update that is already in progress on the SLB9670.
key_group_id	uint32_t	Key group identifier of the SLB9672. It is used to select the correct manifest from the firmware image.
firmware_update_properties	uint32_t	Firmware update capability flags reported by the SLB9672. The updater does not use this value for the update decision.
start_hash_algorithm	uint16_t	Hash algorithm used for the stored start hash of an SLB9672 update.
start_hash_size	uint16_t	Size of the start hash stored in start_hash_digest.
start_hash_digest	uint8_t[64]	Hash of the manifest with which an SLB9672 firmware update was started.
vendor_string	uint8_t[16]	16 byte TPM vendor string.

Table 1: Variables of the TPMInformation structure.

The TPMInformation structure is used to bundle information about the TPM together and pass it to the functions as one parameter. This should reduce calls to the TPM to check properties. Some variables are only available on the SLB9670 or SLB9672.

3.2.1 Standard TPM Properties

Property	Value	Meaning
TPM2_PT_MANUFACTURER	0x0105	TPM manufacturer identifier
TPM2_PT_VENDOR_STRING_1 to TPM2_PT_VENDOR_STRING_4	0x0106 to 0x0109	Contains a vendor specific 16 byte string across four properties.
TPM2_PT_FIRMWARE_VERSION_1	0x010B	Contains the first 4 bytes of the TPM firmware version.
TPM2_PT_FIRMWARE_VERSION_2	0x010C	Contains the second 4 bytes of the TPM firmware version.

Table 2: Standard TPM properties used by the firmware updater

The two firmware version values are combined into an 8 byte firmware_version value. The vendor string is used to identify the TPM type.

3.2.2 Infineon Vendor-Specific Properties

Infineon defines additional TPM properties for information that is not part of the standard TPM 2.0 property set. The vendor properties start at `0x8000_0000`. The constants are taken from the Infineon updater from the file `TPM2_VendorTypes.h`.

Property	Value	TPM	Value returned
`TPM_PT_VENDOR_FIX_SMLI2`	`0x8000_0001`	SLB9670	`sSecurityModuleLogicInfo2_d`
`TPM_PT_VENDOR_FIX_FU_COUNTER`	`0x8000_0003`	SLB9672	16 bit update counter
`TPM_PT_VENDOR_FIX_FU_COUNTER_SAME`	`0x8000_0004`	SLB9672	16 bit same version counter
`TPM_PT_VENDOR_FIX_FU_START_HASH_DIGEST`	`0x8000_0005`	SLB9672	Hash algorithm and digest
`TPM_PT_VENDOR_FIX_FU_OPERATION_MODE`	`0x8000_0007`	SLB9672	8 bit operation mode
`TPM_PT_VENDOR_FIX_FU_KEYGROUP_ID`	`0x8000_0008`	SLB9672	32 bit key group identifier

Table 3: Vendor TPM properties used by the firmware updater

The two TPM generations differ in how they provide vendor specific properties. The SLB9670 returns most firmware update information in the `sSecurityModuleLogicInfo2_d` structure. The SLB9672 has several smaller properties to get the required information from the TPM.

Vendor property data layout The vendor property payload uses the `IFX_Vendor_Property` construct.

```
struct IFX_Vendor_Property{
    uint32_t count;
    uint16_t size;
    uint8_t buffer[TSS_MAX_DIGEST_BUFFER];
};
```

Figure 2: Vendor TPM response structure

Variable	Type	Meaning
`count`	`uint32_t`	Number of returned vendor property values
`size`	`uint16_t`	Size of the following property value in bytes.
`buffer`	`uint8_t[]`	Binary property value.

Table 4: Vendor TPM response

`IFX_Vendor_Property` is a project helper for the packed vendor capability representation.

3.2.3 SLB9670 Security Module Information

The `TPM_PT_VENDOR_FIX_SMLI2` property returns the `sSecurityModuleLogicInfo2_d` structure. The structure is defined in the `TPM2_FieldUpgradeTypes.h` file of the Infineon updater.

```
typedef struct sSecurityModuleLogicInfo2_d{
    uint16_t                internal1;
    uint16_t                wMaxDataSize;
    sSecurityModuleLogic_d  sSecurityModuleLogic;
    uint16_t                SecurityModuleStatus;
    sFirmwarePackage_d      sProcessFirmwarePackage;
    uint16_t                internal6;
    uint8_t                 internal7[6];
    uint16_t                wFieldUpgradeCounter;
    sKeyList_d              sKeyList;
} sSecurityModuleLogicInfo2_d;
```

Figure 3: SLB9670 security module information structure from Infineon

Variable	Type	Meaning and use
`wMaxDataSize`	`uint16_t`	Maximum block size for the transfer.
`sSecurityModuleLogic .unTargetHardwareIdentifier`	`uint32_t`	Hardware identifier used for firmware validation.
`sSecurityModuleLogic .sFirmwareConfiguration`	`sFirmwarePackages_d`	Determines the active firmware package version.
`SecurityModuleStatus`	`uint16_t`	Indicates whether the normal firmware or the bootloader is active.
`sProcessFirmwarePackage .StaleVersion`	`uint32_t`	The lower four bits are used as the unique ID when an update is already in progress.
`wFieldUpgradeCounter`	`uint16_t`	Number of firmware upgrades still available.
`sKeyList.wEntries`	`uint16_t`	Number of valid decryption key identifiers.
`sKeyList.DecryptKeyId`	`uint32_t[4]`	Decryption key identifiers accepted by the TPM.

Table 5: SLB9670 security module information

The fields named `internal*` are part of the Infineon structure, but their meaning is not documented.

3.2.4 Device Identification

There are 3 constants for identifying the Infineon TPMs.

Constant	Value	Meaning
`INFINEON_MANUFACTORY_CODE`	`0x49465800`	Infineon manufacturer value, corresponding to `IFX`
`INFINEON_SLB9670_TYPE`	`0x534C423936373000`	Value used to identify an SLB9670
`INFINEON_SLB9672_TYPE`	`0x534C423936373200`	Value used to identify an SLB9672

Table 6: Device Identification

The manufacturer is used to restrict the updater to TPMs of Infineon, and the types define which update protocol is used.

3.3 Firmware Update Package

The updater needs a binary firmware image from Infineon. The image first has to be parsed into the fields required for compatibility checks and the firmware update.

3.3.1 Firmware Reader

The binary is parsed with the `TPMFirmwareReader` structure.

```
struct TPMFirmwareReader{
    uint8_t *buffer;
    size_t size;
    size_t offset;
};
```

Figure 4: Firmware reader structure

Variable	Type	Meaning and use
`buffer`	`uint8_t *`	Pointer to the binary firmware data.
`size`	`size_t`	Size of the buffer in bytes.
`offset`	`size_t`	Offset in the buffer for the next byte.

Table 7: Variables of the firmware reader

`TPMFirmwareReader` is used to parse a firmware binary. Before a parsing function is performed, the remaining size is checked to make sure that the operation does not read outside the buffer. This prevents a malformed firmware image from causing page faults and prevents values outside the binary from being parsed.

3.3.2 Package Identification

The Infineon image starts with a GUID, which is saved in `TPMFirmwareGuid`.

```
struct TPMFirmwareGuid{
    uint32_t data1;
    uint16_t data2;
    uint16_t data3;
    uint8_t data4[8];
};
```

Figure 5: Firmware image GUID structure

The last byte in `data4` is used to define the image version. The base GUID is taken from the file `FirmwareImage.h` in the Infineon updater. The remaining bytes have to match the base GUID.

3.3.3 Firmware Image Versions

The project supports the same Infineon firmware image versions V1 to V3 that are defined in the Infineon updater.

Image Version	GUID last byte	Main difference used by the updater
V1	`0x93`	Uses the legacy policy.
V2	`0x94`	Uses manifest data and contains a padding field before the manifests.
V3	`0x95`	Uses manifest data and stores the image structure version at the beginning of the image.

Table 8: Infineon firmware image versions

The image version and the image structure version are different values. The image version changes the main binary layout. The image structure version is used for smaller, non breaking additions to the format.

3.3.4 Firmware Image Structure

The parsed firmware image is saved in `TPMFirmwareImage`. The structure is from the Infineon TPM updater, where it is called `IfxFirmwareImage` in `Common/FirmwareImage.h`. Some types were changed to their integer counterparts, and normal char arrays were used instead of the wide character array.

```
struct TPMFirmwareImage{
    TPMFirmwareGuid unique;
    uint8_t bImageVersion;
    uint8_t bSourceTpmFamily;
    uint16_t usSourceVersionsCount;
    uint16_t usSourceVersionsSize;
    uint8_t *prgwszSourceVersions;
    uint8_t bTargetTpmFamily;
    uint16_t usTargetVersionSize;
    char wszTargetVersion[65];

    uint32_t unManifestDataSize;
    uint8_t *rgbManifestData;
    uint16_t usNumManifests;
    uint32_t unKeyGroupId;

    uint16_t usPolicyParameterBlockSize;
    uint8_t *rgbPolicyParameterBlock;

    uint32_t unFirmwareSize;
    uint8_t *rgbFirmware;

    uint16_t usImageStructureVersion;
    uint32_t bfTargetState;
    uint32_t bfCapabilities;
    uint16_t usIntSourceVersionCount;
    uint32_t rgunIntSourceVersions[8];

    uint32_t unChecksum;
    uint16_t usSignatureKeyId;
    uint16_t usSalesCodeSize;
    uint8_t rgbSalesCode[16];
    uint8_t rgbSignature[256];
};
```

Figure 6: Parsed firmware image structure

Variable	Type	Meaning and use
`unique`	`TPMFirmwareGuid`	Image version and image format.
`bImageVersion`	`uint8_t`	Firmware image format version.
`usSourceVersionsCount`	`uint16_t`	Number of source firmware versions contained in the source version list.
`usSourceVersionsSize`	`uint16_t`	Size of the source version data.
`prgwszSourceVersions`	`uint8_t *`	Pointer to the source version data for compatibility checks.
`unManifestDataSize`	`uint32_t`	Size of the manifest data for V2 and V3 images.
`rgbManifestData`	`uint8_t *`	Pointer to the manifest data for the SLB9672.
`usPolicyParameterBlockSize`	`uint16_t`	Size of the policy or manifest block.
`rgbPolicyParameterBlock`	`uint8_t *`	Pointer to the policy block or manifest.
`unFirmwareSize`	`uint32_t`	Size of the firmware.

Variable	Type	Meaning and use
rgbFirmware	uint8_t *	Pointer to the firmware that is transferred to the TPM.
usImageStructureVersion	uint16_t	Version of the firmware image structure fields.
bfCapabilities	uint32_t	Firmware image capability bits.
usIntSourceVersionCount	uint16_t	Number of source versions.
rgunIntSourceVersions	uint32_t[8]	Source versions when an SLB9670 is in bootloader mode.
unChecksum	uint32_t	CRC32 checksum of the firmware image.

Table 9: Variables of the TPMFirmwareImage structure

The pointers used in the structure point into the firmware binary that was read from the file.

3.3.5 Manifest and Policy Data

The parameter block has different meanings depending on the firmware version. V1 uses a legacy policy parameter block. V2 and V3 use manifest data. For the SLB9672, the manifest data can contain more than one manifest. The manifest with the same key group ID as the connected TPM is selected.

Field	Type	Meaning and use
manifest_count	uint16_t	Number of manifests in the container.
key_group_id	uint32_t	Identifies which TPM key group the manifest belongs to.
manifest_size	uint16_t	Size of the following manifest in bytes.
manifest	uint8_t[]	Manifest data sent to the SLB9672.

Table 10: Values in the SLB9672 manifest container

For the SLB9672, the manifest data contains a manifest count followed by the key group ID, size and data of each manifest.
When a matching key group ID is found, the manifest becomes the active rgbPolicyParameterBlock. It is later hashed for the start command and transferred to the TPM.

3.3.6 Firmware Payload

The firmware payload starts with a 32 bit size followed by the data with that size.

```
uint32_t firmware_size;
uint8_t firmware[firmware_size];
```

Figure 7: Firmware payload

The parser stores the size in unFirmwareSize and a pointer to the data in rgbFirmware. The update code later splits this data into blocks that can be transferred to the TPM.

3.4 Firmware Blob Parsing

3.4.1 Serialized Layout

The firmware image is not parsed by copying the complete binary into the structure. The fields are read in the order defined by the Infineon image format because the image is version dependent and has variable length data.

The relevant order is:

1. firmware image GUID
2. image structure version for V3

3. source TPM family and source versions
4. target TPM family and target version
5. policy parameter block or manifest data
6. firmware payload
7. optional additional image data
8. checksum

The size fields are checked before the data is read. This is the case for the source version, target version, parameter block, manifest data, and firmware payload.

3.4.2 Variable-Length Data

The source versions are stored as a UTF-16 multi string. Each version is zero terminated, and the complete list has an additional zero terminator.
The target version is also stored as a UTF-16 string. The project converts the UTF-16 string into a character string during parsing. The source version data is kept as a pointer because it is later used for the compatibility check.

3.4.3 SLB9670 Legacy Policy Data

V1 images contain a signed policy parameter block. The Infineon format contains several nested structures with many fields that are named `internal`. The project has to parse these fields to keep the correct position in the binary, but they are not used by the updater. For this reason, only the structures and values that are used by the current update validation are described.

`sMessageDigest_d`

```
typedef struct sMessageDigest_d{
    uint16_t wSize;
    uint8_t rgbMessageDigest[32];
} sMessageDigest_d;
```

Figure 8: Message digest structure from Infineon

Variable	Type	Meaning and use
`wSize`	`uint16_t`	Size of the firmware digest.
`rgbMessageDigest`	`uint8_t[32]`	SHA-256 digest compared with the calculated firmware digest.

Table 11: Variables of `sMessageDigest_d`

`sFirmwarePackage_d`

```
typedef struct sFirmwarePackage_d{
    uint32_t FwPackageIdentifier;
    uint32_t Version;
    uint32_t StaleVersion;
} sFirmwarePackage_d;
```

Figure 9: Firmware package structure from Infineon

Variable	Type	Meaning and use
`Version`	`uint32_t`	Firmware package version. Used to check which SLB9670 firmware package is active.
`StaleVersion`	`uint32_t`	Used to determine the active firmware package and obtain the unique ID of an update in progress.

Table 12: Variables of `sFirmwarePackage_d`

The `FwPackageIdentifier` is not used in the updater.

`sVersions_d`

```
typedef struct sVersions_d{
    uint32_t internal1;
    uint16_t wEntries;
    uint32_t Version[8];
} sVersions_d;
```

Figure 10: Integer source version structure from Infineon

Variable	Type	Meaning and use
`wEntries`	`uint16_t`	Number of valid integer source versions.
`Version`	`uint32_t[8]`	Source versions used when an SLB9670 is already in the bootloader.

Table 13: Variables of `sVersions_d`

The larger `sSignedData_d`, `sSignerInfo_d`, and `sSignedAttributes_d` structures are used as containers around these values. `sSignedAttributes_d.DecryptKeyId` is used by the updater for the decryption key check. Most of the remaining fields are named `internal` in the Infineon source and are not used in the project.

3.4.4 Byte Order and Bounds Checking

The values in the firmware binary are read in big endian order. The parser reads the values byte by byte and combines them into the correct value. The UTF-16 strings are handled separately in little endian order. Only letters, digits, underscores, and dots are accepted for the version strings. Before each read, the remaining buffer size is checked to be large enough to prevent out of bounds reads.

3.5 Firmware Update Commands

3.5.1 Command and Response Format

The normal TPM 2.0 command header contains a tag, command size, and command code. The Infineon vendor commands use the same TPM command transport but define their own command codes and parameters.

```
uint16_t tag;
uint32_t command_size;
uint32_t command_code;
uint8_t  parameters[];
```

Figure 11: General TPM 2.0 command layout used by the vendor commands

Values are serialized in big endian order. The response contains the TPM response code followed by command specific data when the command succeeds.

3.5.2 SLB9670 Commands

Constant	Value	Meaning
TPM2_CC_Field_Upgrade_Start_Vendor	0x2000012F	Infineon vendor command used to start the firmware update.
TPM20_FieldUpgradeStart	0x34	Start field upgrade.
SLB9670_FU_COMMAND_TAG	0xC1	Command tag used for the field upgrade.
SLB9670_FU_COMMAND_CODE	0xAA	Command code used for the field upgrade.
SLB9670_FU_INFO_REQUEST2	0x11	Returns firmware update information including the maximum block size and current state.
SLB9670_FU_UPDATE	0x31	Transfers one firmware block.
SLB9670_FU_COMPLETE	0x32	Completes the firmware transfer.

Table 14: SLB9670 vendor firmware commands

Firmware blocks are limited by `max_block_size`. Each update block contains an LRC value calculated over the field upgrade subcommand, size, and payload.

3.5.3 SLB9672 Vendor Commands

The SLB9672 uses different vendor TPM 2.0 commands for the manifest, firmware data, and finalization.

Constant	Value	Meaning
TPM20_Field_UpgradeStartManifestHash	0x01	SLB9672 start field upgrade.
TPM2_CC_Field_Upgrade_Start_Vendor	0x2000012F	Starts the update using the manifest hash.
TPM2_CC_Field_Upgrade_Manifest_Vendor	0x20000131	Transfers the selected manifest.
TPM2_CC_Field_Upgrade_Data_Vendor	0x20000132	Transfers firmware data.
TPM2_CC_Field_Upgrade_Finalize_Vendor	0x20000133	Finalizes the firmware update.

Table 15: SLB9672 vendor firmware commands

The data is sent in blocks. The SLB9672 has a maximum block size of 1024 bytes.

3.5.4 Manifest Block Processing

The manifest command includes a processing value that identifies the first, consecutive, or last block.

Constant	Value	Meaning
TPM20_FU_PROCESSING_FIRST_BLOCK	0x01	First block of the manifest.
TPM20_FU_PROCESSING_CONSECUTIVE_BLOCK	0x02	Block when more data follows.
TPM20_FU_PROCESSING_LAST_BLOCK	0x00	Last block of the manifest or no chaining.

Table 16: SLB9672 manifest block processing values

3.6 Firmware Update State Machine

The firmware update state has to be checked during the update because the TPM can change mode or restart during the procedure. The states are different for the two TPMs.

3.6.1 SLB9670 States

The SLB9670 uses two state values that are relevant.

Constant	Value	Meaning
SMS_FWCONFIG_ACTIVE	0x36A5	Normal firmware state.
SMS_BTLDR_ACTIVE	0x5A3C	Field upgrade bootloader is active.

Table 17: SLB9670 firmware states

A normal update starts with the SMS_FWCONFIG_ACTIVE state. After the start command, the TPM changes to the SMS_BTLDR_ACTIVE state. The firmware blocks are then transferred and the update is completed.

3.6.2 SLB9672 Operation Modes

The SLB9672 provides a more detailed operation mode.

Constant	Value	Meaning
TPM_OPERATION_MODE_NORMAL	0x00	TPM is in normal mode.
TPM_OPERATION_MODE_UPDATE_CAN_ABANDON	0x01	Update is active and can still be abandoned.
TPM_OPERATION_MODE_UPDATE_CANNOT_ABANDON	0x02	Update is active and cannot be abandoned.
TPM_OPERATION_MODE_BEFORE_FINALIZE	0x03	Firmware data was transferred to the TPM and the update can be finalized.
TPM_OPERATION_MODE_RESTART_REQUIRED	0x04	A restart is required before the update can continue.
TPM_OPERATION_MODE_RECOVERY_CAN_ABANDON	0x81	Recovery update is active and can still be abandoned.
TPM_OPERATION_MODE_RECOVERY_CANNOT_ABANDON	0x82	Recovery update is active and cannot be abandoned.
TPM_OPERATION_MODE_RECOVERY_BEFORE_FINALIZE	0x83	Recovery update can be finalized.
TPM_OPERATION_MODE_RECOVERY_RESTART_REQUIRED	0x84	Recovery update requires a restart.

Table 18: SLB9672 operation modes

When the TPM is in the normal mode, an update can be started. If an update is already in progress before the finalize stage, the updater selects the manifest for the TPM key group and compares its hash with the stored start hash.

4 Design

This chapter describes the design of the firmware updater. The updater has three parts. The first part handles retrieving information from the TPM that is needed for the update. The second part handles the parsing of the firmware image and checking that the binary format is valid. The last part checks that the firmware image can be applied to the current TPM, checks TPM values, and performs the TPM specific update sequence.
The SLB9670 and SLB9672 share the same general update sequence, but use different vendor commands and update states. For that reason, most of the functions, such as those for firmware parsing, validation and TPM information retrieval, work for both TPMs. The device-specific command sequences are separated into functions for the TPMs.
A firmware update changes the state of the TPM. The updater does not assume that the TPM is always in the normal operating mode. The state is therefore checked before the update is started and while in the update sequence. When an update has already started, it can be continued in states supported by the updater.

4.1 Overview

The program first initializes the TPM connection and retrieves the information required for the firmware update. Afterwards, the firmware image is loaded into memory and parsed into a representation that is used by the updater. The retrieved TPM information and the parsed image are then checked for compatibility between the TPM and the firmware image.
If the check is successful, the updater checks if the TPM is already in an update state that can be continued or if a new update has to be started. For a new update, the platform policy is prepared and the Infineon field upgrade start command is sent. If an update is already active, the start step is skipped. The updater then continues with the transfer and finalization steps required for the detected TPM.

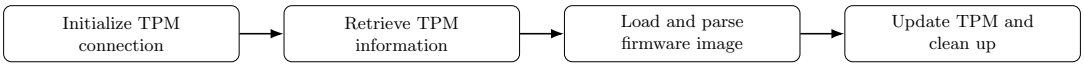


Figure 12: Main program flow

4.2 TPM Communication

The updater uses two different libraries to communicate with the TPM. Standard TPM 2.0 commands are accessed through ESAPI. This includes reading TPM properties, getting random numbers, and creating a policy session.
The Infineon firmware update commands are vendor specific and not always available through the ESAPI interface. These commands are serialized and sent through TCTI. Both libraries use the same TCTI connection to the TPM. Therefore, the updater can use the ESAPI functions and the raw TCTI connection at the same time.

4.2.1 TPM Connection Context

The state required for communication is stored in the `TPMConnection` structure. It contains the memory buffer of the TCTI implementation, the TCTI context, and the ESAPI context. These contexts are used to communicate with the TPM.
The TCTI context is initialized first and is then used to initialize ESAPI. The `TPMConnection` structure is given to different functions that require communication instead of passing the contexts individually to each one. After the update, the connection has to be cleaned up by first finalizing the ESAPI context, then the TCTI context, and finally freeing the TCTI buffer.

4.2.2 TPM Information Context

The information required for the validation and update sequence is stored in `TPMInformation`. This makes it simpler to pass it around in functions. The structure contains the standard TPM properties and device specific values.
The function `tpm_get_information()` fills the structure by calling the individual functions. These functions return the TPM value or a default value if it is not available for the TPM type.

Some values, such as the manufacturer, TPM type, firmware version, update counter, and operation mode, are used by the update logic to determine if the TPM can be updated. Other values are TPM specific and are not available on both TPMs.
The operation mode is not treated as a fixed value because it changes during the update.

4.3 Firmware Parser Design

The firmware parser converts the binary Infineon firmware image into the values needed for validation and the firmware transfer. The complete file is loaded into memory. A `TPMFirmwareReader` is then used to move through the memory to parse the individual fields.
The parser supports the Infineon image versions V1 to V3. The image version decides how the image has to be parsed. The V1 image contains the legacy policy parameter block. The V2 and V3 images contain manifest data.
The parser is kept separate from the device compatibility checks. The parsed image is saved in the structure `TPMFirmwareImage` to be used by the updater. The parser also checks if the parsed image is malformed. Checks that require information from the connected TPM are performed afterwards by the updater.

4.3.1 Parsed Firmware Representation

The parsed image is stored in `TPMFirmwareImage`. The structure is used for all supported firmware image versions.
Large variable length fields are not copied into newly allocated buffers. The pointers point to data in the original firmware buffer. For this reason, the original firmware buffer has to remain allocated as long as the parsed image is used.
The `TPMFirmwareImageFile` structure keeps the parsed image together with the original firmware buffer and its size.

4.3.2 Firmware Reader

`TPMFirmwareReader` contains a pointer to the buffer, the buffer size, and the current offset into the buffer. The parser uses this structure instead of directly increasing the raw pointers. Before a value is read, skipped, or returned as a pointer, the remaining size is checked.
Reader functions are available for 8 bit, 16 bit, and 32 bit values, arrays, pointers, skipped bytes, and UTF-16 strings. Integer values are read in the byte order required by the firmware. The version strings are handled separately and converted from UTF-16 to char.

4.3.3 Validation Strategy

Validation is done in two parts, during parsing and during the update. During parsing, the updater checks that the image uses the expected format from Infineon, such as the image version and GUID. Before every read, the remaining buffer is checked to contain enough data. For strings, the parser checks that the size is valid and that the characters are supported.
After the image parsing, the updater checks if the TPM can be updated using the TPM information and if the firmware image can be used. TPM values such as manufacturer, type and remaining update counter are checked. For the image, the CRC32 value is calculated and compared with the stored checksum to check that the image is not corrupted.
For the SLB9670, the source version is checked against the current firmware version. When the TPM is in the bootloader state, the active firmware version is checked against the integer source versions if an active firmware version is available. The target hardware identifier reported by the TPM is checked against the SLB9670 identifier used by the updater. The legacy policy data is parsed again and the SHA-256 digest of the firmware payload is compared with the message digest from the signed attributes. The required decryption key ID also has to be present in the list reported by the TPM. If the image requires the firmware unique ID to match while an update is already running, this value is checked as well.
For the SLB9672, the same version update counter has to be greater than zero and the operation mode has to be one of the modes supported by the current update handling. Restart required modes are rejected and require the TPM to be restarted before the updater is run again. The manifest matching the TPM key group ID is selected and the source version is checked. If the TPM is in an active update mode while the transfer is not yet complete, the updater hashes the selected manifest and compares the result with the start hash stored by the TPM. This prevents an active update from being continued with a different manifest.

4.4 Firmware Updater Design

The updater has a function that handles the whole update procedure for the TPM. It can be called by both TPM generations. It performs the check, start, transfer, and finalization steps. The sequence of the exact commands depends on the TPM type.
Figure 13 shows the complete update flow and the points where the SLB9670 and SLB9672 use different update sequences.

4.4.1 Update Interface

Before a new update is started, the updater prepares the platform policy required for the Infineon field upgrade start command. A policy session is created and restricted to the field upgrade start command. The resulting policy handle is then used for the start command.
The current implementation assumes an empty platform hierarchy authorization. A random nonce from the TPM is used when the policy session is created.
For the SLB9670, the legacy policy parameter block is sent with the start command. For the SLB9672, a SHA-512 hash of the selected manifest is calculated and sent together with the hash algorithm identifier.
The update functions return a boolean status. If a function fails, it returns false and the update sequence stops. Failed firmware blocks are not sent again automatically.

4.4.2 Device-Specific Update Paths

The SLB9670 uses the older field upgrade command sequence. After the start command, the updater waits until the TPM is in the bootloader state. The firmware payload is then split into blocks using the maximum block size that is retrieved from the TPM. Every block is sent with the SLB9670 update subcommand and contains the LRC value. After all blocks have been transferred, the complete command is sent. If the bootloader state is already active when the updater starts, the start command is skipped. The compatibility checks use the active firmware information and, if required, the firmware unique ID before the blocks are transferred.
The SLB9672 uses separate vendor commands for the start, manifest, firmware data, and finalization. After an update is started, the updater waits until the TPM is in the update mode. The selected manifest is sent first. The manifest is split into blocks and each block contains a processing value for the first, consecutive, or last manifest block. Afterwards, the firmware payload is sent in blocks with a maximum size of 1024 bytes. If the TPM is already in a before finalize state, the transfer stage returns without sending the manifest and firmware again. The updater then continues to the finalization stage. The same update path also accepts the corresponding recovery states.

4.4.3 Update State Handling

The updater uses defined TPM states during the update. The updater does not keep a separate persistent update state on the host. It reads the state from the TPM and uses it to decide which update steps have to be performed.
For the SLB9670, a new update starts while the normal firmware is active. After the start command, the updater waits until the TPM enters the bootloader. The firmware blocks are then transferred. If the TPM is already in the bootloader when the updater is started, the start step is skipped and the updater continues with the transfer after the compatibility checks.
For the SLB9672, a new update starts in normal mode. After the start command, the updater waits until the TPM enters an update mode. The manifest and firmware data are then transferred. If the TPM is already in a before finalize state, the transfer is skipped and the updater continues with the finalize command. The corresponding recovery states use the same update path. Restart required states stop the current updater run and require the TPM to be restarted before the updater is run again.

4.5 Error and Recovery Handling

The updater stops when an error occurs. Functions check that arguments and values are valid. If a check fails, false is returned to the caller and the updater stops the update sequence.
The firmware parser checks that enough data remains in the buffer before values are read, skipped, or referenced. Each response returned from the TPM is checked to determine if the command was successful.

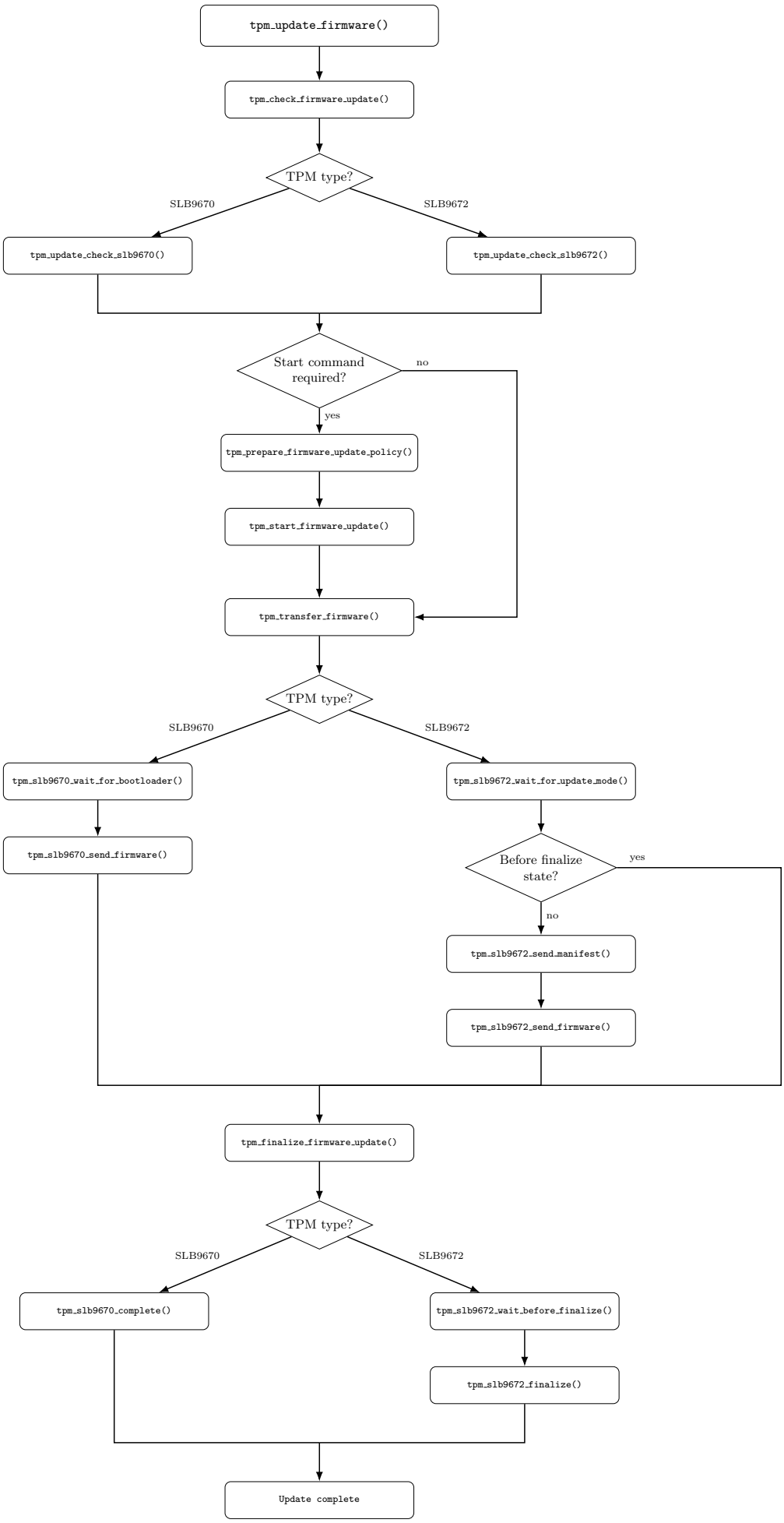


Figure 13: Firmware update control flow for the SLB9670 and SLB9672

For state transitions, there are retries that give the TPM time to transition into the new state. For the SLB9672, the TPM connection is reinitialized during these retries.
Recovery of an interrupted update is based on information stored by the TPM. For the SLB9670, the updater uses the bootloader state and the firmware information of the active update. For the SLB9672, the updater uses the operation mode and, while the transfer is still active, the stored manifest start hash. This allows the updater to detect an update that is already in progress after the program has been restarted.

4.6 Security Considerations

The firmware image is checked before the update is started. The parser checks the remaining buffer size before data is read or referenced, and the updater compares the CRC32 value from the image with a calculated CRC32 value. The device specific checks are then performed before the firmware is transferred.
Starting a new update requires the platform policy and a policy session that is restricted to the Infineon field upgrade start command. The firmware signature is parsed from the image. It is cryptographically verified by the TPM itself. Therefore, the CRC32 check is only used as a corruption check.

5 Implementation

This chapter describes the implementation of the firmware updater in Gapfruit OS. The implementation is separated into TPM communication, TPM information retrieval, firmware parsing, vendor command handling, and the two device specific update sequences.
The updater is implemented in C++ and is split into multiple files. The `tpm2_get_info.*` files contain the functions for retrieving the information from the TPM. The `tpm2_file_parser.*` files contain the firmware file and firmware image parser. The `tpm2_updater.*` files contain the update checks, TPM command serialization, and the firmware update sequence. The `main.cc` file handles initialization and cleanup when the TPM is updated.
The implementation uses ESAPI for standard TPM functionality and TCTI for the commands that are vendor specific and not available through ESAPI. The SLB9670 and SLB9672 share the same connection, parsed firmware, and main update interface. The functions that depend on the update protocol are separated by the TPM type.

5.1 Component Implementation

The entry point of the component first initializes the TPM connection with `tpm_init_connection()`. Afterwards, `tpm_get_information()` retrieves the values required for the firmware update. The firmware image is loaded into memory with `parse_bin()` and parsed by `tpm_parse_firmware_image()`. The loaded image and the parsed `TPMFirmwareImage` are combined in a `TPMFirmwareImageFile`. Multiple pointers in the `TPMFirmwareImage` point to the firmware image. For that reason, the buffer of the image has to remain allocated while the firmware image is used.
The update sequence is implemented in `tpm_update_firmware()`. The function first checks the firmware image and the TPM information. When a new update is started, it creates the required policy and sends the field upgrade start command. Afterwards, the firmware is transferred and the update is finalized. If the TPM is already in an update state supported by the implementation, the start step is skipped and the updater continues the existing update.
In `main.cc`, the TPM initialization, information retrieval, firmware loading, and parsing are combined.

5.1.1 TPM Connection Context

The firmware updater requires TCTI and ESAPI contexts to communicate with the TPM. The two contexts are combined in the `TPMConnection` structure. This allows the connection state to be passed between the functions as one object.
The connection is initialized in `tpm_init_connection()`. `Tss2_Tcti_Device_Init()` is called to retrieve the required size for the TCTI state. A buffer is then allocated with the size and is used as the `TSS2_TCTI_CONTEXT`. The device TCTI is then initialized for `/dev/tpmraw0`.
After the TCTI initialization, the ESAPI context is initialized with `Esys_Initialize()` using the TCTI context. `Esys_Startup()` is then called with `TPM2_SU_CLEAR`. At this point, both libraries use the same TPM connection.
The dependencies between the contexts define the cleanup order. `tpm_clean_up_connection()` first finalizes the ESAPI context with `Esys_Finalize()`. Afterwards, the TCTI context is finalized with `Tss2_Tcti_Finalize()`. Finally, the TCTI buffer and the `TPMConnection` structure are freed.

5.1.2 TPM Information Retrieval

The TPM values used by the updater are retrieved by `tpm_get_information()`. The function allocates a `TPMInformation` structure and calls the individual getter functions. This keeps the device information in the structure, which is later used in the firmware updater check.
Standard values such as the manufacturer, vendor string, firmware version, and TPM type are retrieved with `Esys_GetCapability()`. The firmware version is created from `TPM2_PT_FIRMWARE_VERSION_1` and `TPM2_PT_FIRMWARE_VERSION_2`. The TPM type is created from the first two vendor string properties and is compared with constants.
The device specific information is retrieved with `Esys_GetCapability()` using the Infineon vendor properties. For the SLB9670, several values are part of the `TPM_PT_VENDOR_FIX_SMLI2` property. The implementation uses it to get the upgrade counter, operation mode, maximum block size, target hardware identifier, decryption key IDs, active firmware version, and firmware unique ID.
The SLB9672 provides the values through separate vendor properties. The implementation retrieves the update counter, same version update counter, operation mode, key group ID, firmware

update properties, and the stored start hash. Values which are only available on the other TPM type are initialized with zero. The maximum block size is 1024 bytes.
The returned vendor properties contain their own count and size fields. These are checked before the property data is used.

5.2 Firmware Parser

The firmware parser is implemented in `tpm2_file_parser.cc`. The implementation can load a binary firmware image directly with `parse_bin()`.
`parse_bin()` first reads the size of the file and allocates a buffer for it. The file is then read into this buffer and a `TPMFirmwareReader` is initialized with the buffer, its size, and an offset of zero.
The reader helper functions are used for the parsing. Functions are implemented for 8 bit, 16 bit, and 32 bit values, arrays, pointers into the buffer, skipped data, and UTF-16 strings. Every function checks the remaining size before advancing the offset. This keeps the bounds checking in the reader functions instead of repeating it for every firmware field.
The integer values in the firmware image are read in big endian order. UTF-16 strings are handled separately because their bytes are stored in little endian order. `reader_utf16_string()` converts the supported UTF-16 characters into a normal character string. The implementation only accepts letters, numbers, underscores, and dots for these version strings.
`tpm_parse_firmware_image()` performs the complete image parsing. First, the firmware GUID is read and compared with the Infineon base GUID. The last byte is used to determine the firmware image version. Image versions V1, V2, and V3 are accepted.
After the GUID, the parser reads the source TPM family and source firmware versions. The source versions remain in the original firmware buffer and `prgwszSourceVersions` points to their position. The strings are parsed once to validate their format and determine the number of source versions.
The target TPM family and target version are parsed afterwards. The target version is converted from UTF-16 and stored directly in `wszTargetVersion`.
The handling of the parameter block depends on the firmware image version. For a V1 image, it is kept as `rgbPolicyParameterBlock` and is parsed as the legacy signed policy data. `tpm_parse_signeddata()` reads the nested structures used by the SLB9670, including the message digest, firmware package information, decryption key identifier, and integer source versions.
For V2 and V3 images, the parameter data is stored as manifest data. V2 images contain a padding size at the beginning of the data, which is read and skipped before the actual manifest container. The manifest for the connected SLB9672 is not selected during the initial parsing because the TPM key group ID is required for this step.
After the parameter data, the firmware size is read and `rgbFirmware` is set to a pointer to the firmware payload. The payload is therefore also not copied into a second buffer.
The parser then reads the additional image data when it is available. Depending on the image structure version, this includes fields such as target state, capabilities, integer source versions, sales code, signature key ID, and signature. Finally, the stored CRC32 value is parsed.

5.3 TPM Command Serialization

The Infineon firmware update commands are not sent through normal ESAPI command functions. For that reason, the required commands are manually serialized and transferred with TCTI.
The helper functions `tpm_update_write_uint8()`, `tpm_update_write_uint16()`, and `tpm_update_write_uint32()` write integer values into a command buffer and advance its offset. The 16 bit and 32 bit values are written in big endian order. Before a value is written, the remaining command buffer size is checked.
Helper functions are used for reading values from TPM responses as well. The common function `tpm_update_transmit()` sends the serialized command with `Tss2_Tcti_Transmit()` and receives the result with `Tss2_Tcti_Receive()`.
After receiving the response, the implementation first checks that at least the normal TPM response header is available. The response size stored in the header is then checked against the number of bytes returned by TCTI. Finally, the TPM response code is read. The function only returns successfully when the response code is `TPM2_RC_SUCCESS`.
The updater also contains shared functions for image checks and cryptographic hashes. `tpm_update_crc32()` calculates the CRC32 value used by the firmware image format. `tpm_update_check_image_crc()` calculates the CRC over the firmware image except for the final four checksum bytes and compares it with `unChecksum`.

SHA-256, SHA-384, and SHA-512 are calculated in `tpm_update_hash()` using the crypto library available to the component. SHA-256 is used for the SLB9670 firmware digest. The SLB9672 uses SHA-512 when starting a new update, while SHA-384 or SHA-512 can be used when checking the start hash of an update already in progress.
Before a new update is started, `tpm_prepare_firmware_update_policy()` creates the policy session required by the start command. The current implementation sets an empty authorization for the platform hierarchy. A random nonce is requested from the TPM with `Esys_GetRandom()`, and a policy session is created with `Esys_StartAuthSession()`.
The policy is restricted to the firmware update start command with `Esys_PolicySecret()` and `Esys_PolicyCommandCode()`. The TPM handle of this policy session is then used when the manually serialized `TPM2_CC_Field_Upgrade_Start_Vendor` command is sent.

5.4 SLB9670 Support

The SLB9670 implementation first checks that a V1 firmware image with a policy parameter block is available. The TPM also has to be either in the normal firmware state `SMS_FWCONFIG_ACTIVE` or in the bootloader state `SMS_BTLDR_ACTIVE`.
The target hardware identifier is checked when the TPM reports one. The firmware source version is then compared with the current TPM firmware version. When the TPM is already in the bootloader, the active firmware version is compared with the integer source versions from the firmware image instead.
The legacy policy parameter block is parsed again by `tpm_update_check_slb9670_policy()`. The firmware payload is hashed with SHA-256, and the resulting digest is compared with the message digest from the policy data. The decryption key ID from the policy data also has to be available on the TPM.
When an update is already in progress and the firmware image requires the unique ID check, the lower four bits of `StaleVersion` are used as the firmware image unique ID. This value is compared with the unique ID reported by the TPM.
For a new update, the policy parameter block is passed to `tpm_field_upgrade_start_vendor()` together with the SLB9670 start command. After the start command, `tpm_slb9670_wait_for_bootloader()` repeatedly sends the `InfoRequest2` command until the bootloader state is reported. The response is also used to update the maximum firmware block size.
The firmware payload is transferred by `tpm_slb9670_send_firmware()`. The payload is divided into blocks with the maximum block size returned by the TPM. Each block is serialized as an SLB9670 field upgrade update command.
The block contains an LRC value. `tpm_slb9670_calculate_lrc()` calculates the value by XORing the field upgrade subcommand, the two block size bytes, and the firmware data.
After all firmware blocks are transmitted, `tpm_slb9670_complete()` sends the field upgrade complete command. The response is checked and the updater waits before continuing to give the TPM time to finish the state transition.

5.5 SLB9672 Support

The SLB9672 uses the manifest based update sequence. Before the update, `tpm_select_firmware_manifest()` parses the manifest container. It reads the number of manifests and then iterates through the key group ID and size of each entry. The manifest with the same key group ID as the connected TPM is selected and stored as the active policy parameter block.
The source firmware version and update counters are checked before continuing. The implementation checks the current operation mode. Normal, active, recovery update, and before finalize states are handled. Restart required states are rejected because the TPM has to be restarted before the updater can continue.
To start a new update, the selected manifest is hashed with SHA-512. The algorithm identifier and the 64 byte digest are placed in the start command data and passed to `tpm_field_upgrade_start_vendor()` with `TPM20_FieldUpgradeStartManifestHash`.
Afterwards, `tpm_slb9672_wait_for_update_mode()` waits until the TPM leaves the normal operation mode. During this step, the TPM connection is reinitialized because the existing connection can no longer be used after the state change.
The selected manifest is sent before the firmware payload. `tpm_slb9672_send_manifest()` divides it into blocks with a maximum size of 1024 bytes. Each command contains a processing value which marks the block as the first, consecutive, or last block.

The firmware is then transferred by `tpm_slb9672_send_firmware()`. It is also divided into blocks with a maximum size of 1024 bytes and transferred with the field upgrade data vendor command. If the TPM was already in a before finalize state when the program was started, the manifest and firmware transfer are skipped. This allows an update that has already completed the transfer to continue with finalization.
Before finalizing, `tpm_slb9672_wait_before_finalize()` repeatedly reconnects to the TPM and reads the operation mode. When `TPM_OPERATION_MODE_BEFORE_FINALIZE` or the corresponding recovery mode is reached, `tpm_slb9672_finalize()` sends the field upgrade finalize vendor command.
An additional `tpm_slb9672_abandon_firmware_update()` function is implemented for abandoning an SLB9672 update. It sends the field upgrade abandon vendor command and reports that a hardware reset or power cycle is required afterwards.

5.6 Integration into Gapfruit OS

The firmware updater is built as a Genode component named `tpm2_update_firmware`. The `target.mk` file includes `main.cc`, `tpm2_file_parser.cc`, `tpm2_get_info.cc`, and `tpm2_updater.cc` as the source files of the component.
The component links against the Genode `base`, `posix`, and `stdcxx` libraries together with `libc`, `tpm2-tss`, and `libcrypto`. `tpm2-tss` provides the ESAPI and TCTI interfaces used for TPM communication. The crypto library is used for the SHA-256, SHA-384, and SHA-512 calculations performed during the firmware checks and update procedure.
The TPM connection uses the device TCTI with `/dev/tpmraw0`. This accesses the TPM before the mux resource handler. The firmware update requires exclusive access to the TPM during the update procedure. In the current Gapfruit OS architecture, the TPM cannot be deactivated for the other components after they have received access to it. For that reason, the firmware updater has to run in a dedicated update mode where the TPM is not made available to the normal system components.

6 Evaluation

The implementation was tested on an IOT-GATE-IMX8PLUS Industrial IoT Gateway equipped with an SLB9670 and a conga-SMX8-Plus equipped with an SLB9672. The goal of the evaluation was to verify that the firmware updater can retrieve the required TPM information, parse the Infineon firmware image, perform the compatibility checks, and execute the complete firmware update procedure on Gapfruit OS.

For both development boards, the updater was first used to retrieve the TPM properties required for the firmware update. This included the manufacturer, TPM type, firmware version, update counters, and the device specific firmware update information. The retrieved information was then used by the compatibility checks before starting the update.

The firmware parser was evaluated with the firmware images used for the two development boards. The images were successfully parsed into the internal `TPMFirmwareImage` representation. During parsing, the image GUID, image version, sizes of variable-length fields, and firmware structure were checked. The updater additionally calculates the CRC32 checksum of the firmware image and compares it with the checksum contained in the image. During development, an issue in the parser caused one of these checks to fail, and the update was stopped before the firmware transfer started. This showed that the validation prevented the updater from continuing with an incorrectly parsed firmware image.

The device specific validation was also exercised as part of the firmware updates. For the IOT-GATE-IMX8PLUS Industrial IoT Gateway, the updater checks the source firmware version, target hardware identifier, available update counter, required decryption key, and the SHA-256 digest contained in the legacy policy data. For the conga-SMX8-Plus, the updater selects the manifest matching the TPM key group ID, checks the source version and update counters, and verifies that the current operation mode can be handled by the updater.

A complete firmware update was successfully performed on the IOT-GATE-IMX8PLUS Industrial IoT Gateway. The updater started the field upgrade procedure, detected the transition into the bootloader, transferred the firmware using the maximum block size reported by the TPM, and completed the update with the field upgrade complete command. After the update, the reported firmware version was newer than before the update and the firmware update counter had decreased. This confirms that the update was applied by the TPM.

A complete firmware update was also successfully performed on the conga-SMX8-Plus. The updater selected the manifest belonging to the TPM, started the update using the manifest hash, transferred the manifest and firmware data, waited for the required operation modes, and finalized the firmware update. After the update, the reported firmware version was newer than before the update and the firmware update counter had decreased. This confirms that the firmware update was completed successfully on the SLB9672 as well.

The evaluation shows that the common implementation can support both TPM generations while keeping the device-specific parts of the update sequence separated. TPM information retrieval, firmware parsing, compatibility validation, vendor command serialization, firmware transfer, and finalization are performed by the Genode component without relying on the original Linux environment of the Infineon reference updater.

7 Conclusion

7.1 Summary of Results

The goal of this project was to implement a firmware updater for the Infineon SLB9670 and SLB9672 TPMs on Gapfruit OS. The Infineon reference updater was analyzed to understand the required firmware image format, TPM properties, vendor specific commands, and update sequences.
Based on this information, a Genode component was implemented that retrieves the required TPM information, parses and validates the firmware image, and performs the firmware update. The implementation supports both TPM generations while keeping the device-specific update procedures separated.
Firmware updates were successfully performed on both supported TPMs. After the updates, the TPMs reported a newer firmware version and a decreased firmware update counter. Therefore, the main goal of the project was achieved.

7.2 Limitations

The updater currently supports the Infineon SLB9670 and SLB9672 TPMs used during the project. Other Infineon TPMs are not supported.
Because of the way TPM resources are currently handled in Gapfruit OS, the updater cannot be used during normal system operation. When the TPM is configured to run in bootloader mode, it cannot provide the same functionality as in normal mode. Since the TPM cannot be deactivated for the other components when required by the firmware update, the firmware updater requires a separate update mode in which the TPM can be used exclusively for the update procedure.
The implementation was developed and tested with the TPM states and firmware images available during the project.
The current implementation assumes an empty platform hierarchy authorization when the policy session for starting a firmware update is created. Systems using a different platform hierarchy authorization would require additional handling.
The firmware signature contained in the Infineon image is parsed but is not cryptographically verified by the updater. The CRC32 check therefore only detects corruption of the firmware image and does not provide cryptographic verification of the image.

7.3 Future Work

A main future improvement is the integration of a dedicated TPM firmware update mode into Gapfruit OS. This would allow the TPM to be made available exclusively to the firmware updater while the update is performed.
Another important improvement is the enhancement of the TPM MUX resource manager of Gapfruit OS in order to support the vendor-specific interface required to perform firmware updates. However, it is at this stage unclear if this is really necessary, as a device would reboot into a specific firmware update mode where the TPM MUX resource manager would not necessarily be required.
The firmware updater could also be extended to support firmware upgrades for TPMs from other vendors.

References

[1] Intel Corporation. *VRT0010, Intel Chipset Firmware Advisory*. INTEL-SA-01372. CVE-2026-6726, Accessed 25 August 2026. Intel Corporation. URL: https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-01372.html.

[2] Intel Corporation. *VRT0011, Intel Chipset Firmware Advisory*. INTEL-SA-01371. CVE-2026-6727, Accessed 25 August 2026. Intel Corporation. URL: https://www.intel.com/content/www/us/en/security-center/advisory/intel-sa-01371.html.

[3] Advanced Micro Devices. *Trusted Platform Module (TPM) Reference Code Errata*. AMD-SB-7064. CVE-2026-6726 and CVE-2026-6727, Accessed 25 August 2026. Advanced Micro Devices. URL: https://www.amd.com/en/resources/product-security/bulletin/amd-sb-7064.html.

[4] Trusted Computing Group. *Trusted Platform Module 2.0 Library Part 1: Architecture*. Version 185. Accessed 14 July 2026. Trusted Computing Group. URL: https://trustedcomputinggroup.org/wp-content/uploads/Trusted-Platform-Module-2.0-Library-Part-1-Architecture_Version-185_pub.pdf.

[5] Trusted Computing Group. *Trusted Platform Module 2.0 Library Part 2: Structures*. Version 185. Accessed 14 July 2026. Trusted Computing Group. URL: https://trustedcomputinggroup.org/wp-content/uploads/Trusted-Platform-Module-2.0-Library-Part-2-Structures_Version-185_pub.pdf.

[6] Trusted Computing Group. *Trusted Platform Module 2.0 Library Part 3: Commands*. Version 185. Accessed 14 July 2026. Trusted Computing Group. URL: https://trustedcomputinggroup.org/wp-content/uploads/Trusted-Platform-Module-2.0-Library-Part-3-Commands_Version-185_pub.pdf.

[7] Infineon Technologies AG. *TPM Factory Update Tool for Linux / Source code*. URL: https://www.infineon.com/gated/infineon-infineon-tpm-factoryupdatetool-for-linux-v2.03-sourcecode_bde67140-af7a-418b-b2d5-6e7211603efc (visited on 09/25/2026).

[8] Infineon Technologies. *Datasheet TPM SLB9670*. OPTIGA™ TPM SLB 9670 TPM2.0. Accessed 14 July 2026. Infineon Technologies. URL: https://www.infineon.com/assets/row/public/documents/30/49/infineon-slb-9670vq2.0-datasheet-en.pdf?fileId=5546d4626fc1ce0b016fc78270350cd6.

[9] Infineon Technologies. *Datasheet TPM SLB9672*. OPTIGA™ TPM SLB9672 TPM 2.0 FW16.xx. Accessed 7 July 2026. Infineon Technologies. URL: https://www.infineon.com/assets/row/public/documents/30/49/infineon-slb9672-tpm20-spi-fw16.xx-ds-rev1-3-2024-11-18-datasheet-en.pdf?fileId=8ac78c8c93dda25b01944055db1b5d0e.